F.M.L. Assignment 2

Christine Donovan

2026-10-03

Assignment 2: k-NN Classification for Universal Bank

This analysis uses k-nearest neighbors (k-NN) to predict whether a Universal Bank customer will accept a personal loan offer, based on customer demographic and account data.

Load libraries

```r
library(class)
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```r
library(ggplot2)
library(tidyr)
```

Load the raw data UniversalBank.csv file

```r
bank<-read.csv("UniversalBank.csv")
```

Convert Education to a factor

```r
bank$Education <- factor(bank$Education)
#Build and apply dummy-variable recipie
dummy_blueprint <- dummyVars(~ Education, data = bank)
Education_dummies <- predict(dummy_blueprint, newdata = bank)
```

```r
head(Education_dummies)
```

```
##   Education.1 Education.2 Education.3
## 1           1           0           0
## 2           1           0           0
## 3           1           0           0
## 4           0           1           0
## 5           0           1           0
## 6           0           1           0
```

Bank and Education_dummies (with the three new ones) using cbind to combine into a single dataframe.

```r
bank_full<-cbind(bank, Education_dummies)
```

Remove columns: Education, ID and Zip Code

```r
which(names(bank_full) %in% c("ID", "ZIP.Code", "Education"))
```

```
## [1] 1 5 8
```

```r
bank_full<- bank_full[ , -c(1, 5, 8)]
```

Partioning the data training (60%) and the validation set(40%)

```r
set.seed(123)
train_index <- sample(1:nrow(bank_full), 0.60 * nrow(bank_full))
length(train_index)
```

```
## [1] 3000
```

```r
train_data <- bank_full[train_index, ]
nrow(train_data)
```

```
## [1] 3000
```

Partioning the validation set (40%) All rows except the train_index

```r
validation_data<-bank_full[-train_index, ]
nrow(validation_data)
```

```
## [1] 2000
```

Separate predictors from target variables before normalizing

```r
train_data_predictors <- train_data[ , -which(names(train_data) %in% c("Personal.Loan"))]
ncol(train_data_predictors)
```

```
## [1] 13
```

Build and apply Normalization recipe

```r
norm_recipie <- preProcess(train_data_predictors, method = c("range"))
train_predictors <- predict(norm_recipie, newdata = train_data_predictors)
validation_predictors <- predict(norm_recipie, newdata = validation_data[ , -which(names(validation_data) %in%
c("Personal.Loan"))])
summary(train_predictors)
```

```
##       Age           Experience         Income           Family      
##  Min.   :0.0000   Min.   :0.0000   Min.   :0.0000   Min.   :0.0000  
##  1st Qu.:0.2955   1st Qu.:0.2826   1st Qu.:0.1435   1st Qu.:0.0000  
##  Median :0.5227   Median :0.5000   Median :0.2593   Median :0.3333  
##  Mean   :0.5097   Mean   :0.5045   Mean   :0.3028   Mean   :0.4663  
##  3rd Qu.:0.7273   3rd Qu.:0.7174   3rd Qu.:0.4028   3rd Qu.:1.0000  
##  Max.   :1.0000   Max.   :1.0000   Max.   :1.0000   Max.   :1.0000  
##      CCAvg           Mortgage       Securities.Account   CD.Account   
##  Min.   :0.0000   Min.   :0.00000   Min.   :0.000      Min.   :0.000  
##  1st Qu.:0.0700   1st Qu.:0.00000   1st Qu.:0.000      1st Qu.:0.000  
##  Median :0.1500   Median :0.00000   Median :0.000      Median :0.000  
##  Mean   :0.1924   Mean   :0.08707   Mean   :0.103      Mean   :0.059  
##  3rd Qu.:0.2600   3rd Qu.:0.15433   3rd Qu.:0.000      3rd Qu.:0.000  
##  Max.   :1.0000   Max.   :1.00000   Max.   :1.000      Max.   :1.000  
##      Online         CreditCard      Education.1     Education.2   
##  Min.   :0.0000   Min.   :0.0000   Min.   :0.000   Min.   :0.000  
##  1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:0.000   1st Qu.:0.000  
##  Median :1.0000   Median :0.0000   Median :0.000   Median :0.000  
##  Mean   :0.5997   Mean   :0.2943   Mean   :0.424   Mean   :0.285  
##  3rd Qu.:1.0000   3rd Qu.:1.0000   3rd Qu.:1.000   3rd Qu.:1.000  
##  Max.   :1.0000   Max.   :1.0000   Max.   :1.000   Max.   :1.000  
##   Education.3   
##  Min.   :0.000  
##  1st Qu.:0.000  
##  Median :0.000  
##  Mean   :0.291  
##  3rd Qu.:1.000  
##  Max.   :1.000  
```

Build and separate Label vectors

```r
train_labels <- factor(train_data$Personal.Loan)
validation_labels <- factor(validation_data$Personal.Loan)
```

```r
summary(train_predictors)
```

```
##       Age           Experience         Income           Family      
##  Min.   :0.0000   Min.   :0.0000   Min.   :0.0000   Min.   :0.0000  
##  1st Qu.:0.2955   1st Qu.:0.2826   1st Qu.:0.1435   1st Qu.:0.0000  
##  Median :0.5227   Median :0.5000   Median :0.2593   Median :0.3333  
##  Mean   :0.5097   Mean   :0.5045   Mean   :0.3028   Mean   :0.4663  
##  3rd Qu.:0.7273   3rd Qu.:0.7174   3rd Qu.:0.4028   3rd Qu.:1.0000  
##  Max.   :1.0000   Max.   :1.0000   Max.   :1.0000   Max.   :1.0000  
##      CCAvg           Mortgage       Securities.Account   CD.Account   
##  Min.   :0.0000   Min.   :0.00000   Min.   :0.000      Min.   :0.000  
##  1st Qu.:0.0700   1st Qu.:0.00000   1st Qu.:0.000      1st Qu.:0.000  
##  Median :0.1500   Median :0.00000   Median :0.000      Median :0.000  
##  Mean   :0.1924   Mean   :0.08707   Mean   :0.103      Mean   :0.059  
##  3rd Qu.:0.2600   3rd Qu.:0.15433   3rd Qu.:0.000      3rd Qu.:0.000  
##  Max.   :1.0000   Max.   :1.00000   Max.   :1.000      Max.   :1.000  
##      Online         CreditCard      Education.1     Education.2   
##  Min.   :0.0000   Min.   :0.0000   Min.   :0.000   Min.   :0.000  
##  1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:0.000   1st Qu.:0.000  
##  Median :1.0000   Median :0.0000   Median :0.000   Median :0.000  
##  Mean   :0.5997   Mean   :0.2943   Mean   :0.424   Mean   :0.285  
##  3rd Qu.:1.0000   3rd Qu.:1.0000   3rd Qu.:1.000   3rd Qu.:1.000  
##  Max.   :1.0000   Max.   :1.0000   Max.   :1.000   Max.   :1.000  
##   Education.3   
##  Min.   :0.000  
##  1st Qu.:0.000  
##  Median :0.000  
##  Mean   :0.291  
##  3rd Qu.:1.000  
##  Max.   :1.000  
```

```r
train_data_predictors <- train_data[ , -which(names(train_data) %in% c("Personal.Loan"))]
ncol(train_data_predictors)
```

```
## [1] 13
```

Build data frame for new customer and normalize it

```r
new_customer_q1 <- data.frame(
  Age = 40,
  Experience = 10,
  Income = 84,
  Family = 2,
  CCAvg = 2,
  Mortgage = 0,
  Securities.Account = 0,
  CD.Account = 0,
  Online = 1,
  CreditCard = 1,
  Education.1 = 0,
  Education.2 = 1,
  Education.3 = 0
)
```

```r
new_customer_q1
```

```
##   Age Experience Income Family CCAvg Mortgage Securities.Account CD.Account
## 1  40         10     84      2     2        0                  0          0
##   Online CreditCard Education.1 Education.2 Education.3
## 1      1          1           0           1           0
```

Normalize new customer

```r
new_customer_norm <- predict(norm_recipie, newdata = new_customer_q1)
new_customer_norm
```

```
##         Age Experience    Income    Family CCAvg Mortgage Securities.Account
## 1 0.3863636  0.2826087 0.3518519 0.3333333   0.2        0                  0
##   CD.Account Online CreditCard Education.1 Education.2 Education.3
## 1          0      1          1           0           1           0
```

Problem 1: Run k-NN with k=1. The classification result (0, not accepted)

```r
knn_result_q1 <- knn(train = train_predictors, test = new_customer_norm, cl = train_labels, k = 1)
knn_result_q1
```

```
## [1] 0
## Levels: 0 1
```

Problem 2. Step 1: Rebuild a combined training datafame (normalized predictors + labels) & run check

```r
train_knn_data <- data.frame(train_predictors, Personal.Loan = train_labels)
str(train_knn_data)
```

```
## 'data.frame':    3000 obs. of  14 variables:
##  $ Age               : num  0.6591 0.8864 0.0455 0.9318 0.9773 ...
##  $ Experience        : num  0.674 0.891 0.087 0.891 0.978 ...
##  $ Income            : num  0.0694 0.2037 0.4167 0.3287 0.4028 ...
##  $ Family            : num  0.667 1 0 0.333 0.333 ...
##  $ CCAvg             : num  0.04 0.13 0.54 0.28 0 0.36 0.01 0.29 0.09 0.15 ...
##  $ Mortgage          : num  0 0 0 0.282 0 ...
##  $ Securities.Account: num  0 0 0 0 0 0 0 0 1 0 ...
##  $ CD.Account        : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ Online            : num  1 1 1 0 1 0 1 1 1 0 ...
##  $ CreditCard        : num  0 1 0 0 0 0 1 0 0 1 ...
##  $ Education.1       : num  1 0 1 1 0 0 0 0 0 0 ...
##  $ Education.2       : num  0 1 0 0 0 0 1 0 0 1 ...
##  $ Education.3       : num  0 0 0 0 1 1 0 1 1 0 ...
##  $ Personal.Loan     : Factor w/ 2 levels "0","1": 1 1 1 1 1 1 1 2 1 1 ...
```

Step 2. Set up train() to search for the best K.

```r
Personal.Loan ~.
```

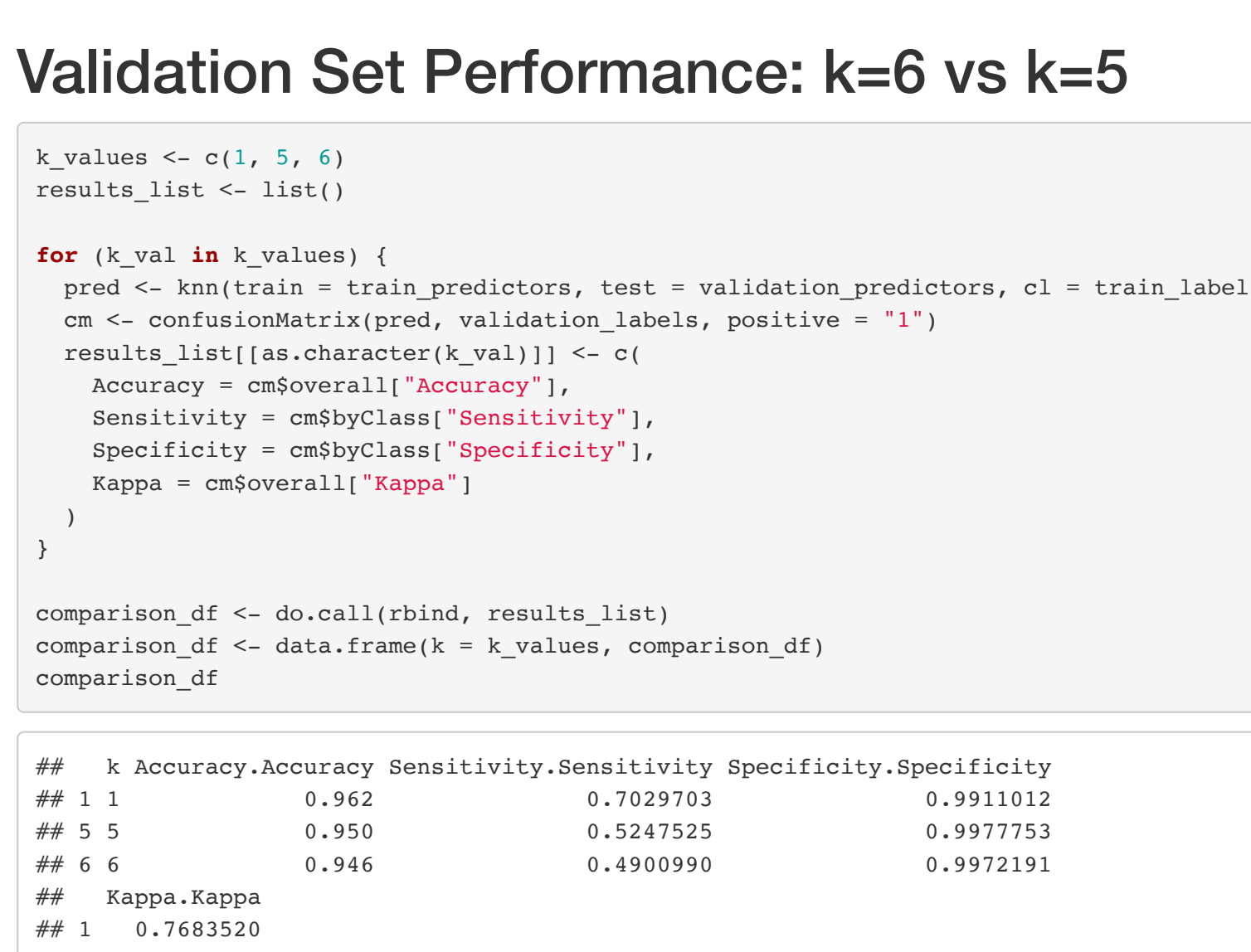
```
## Personal.Loan ~ .
```

```r
knn_tune <- train(Personal.Loan ~ ., data = train_knn_data, method = "knn", tuneGrid = expand.grid(k = 1:20))
knn_tune
```

```
## k-Nearest Neighbors 
## 
## 3000 samples
##   13 predictor
##    2 classes: '0', '1' 
## 
## No pre-processing
## Resampling: Bootstrapped (25 reps) 
## Summary of sample sizes: 3000, 3000, 3000, 3000, 3000, 3000, ... 
## Resampling results across tuning parameters:
## 
##   k   Accuracy   Kappa    
##    1  0.9523785  0.6838790
##    2  0.9496375  0.6579394
##    3  0.9490091  0.6442402
##    4  0.9470101  0.6232175
##    5  0.9474964  0.6171187
##    6  0.9455964  0.5962091
##    7  0.9457467  0.5922486
##    8  0.9441096  0.5750810
##    9  0.9432754  0.5645698
##   10  0.9411579  0.5437132
##   11  0.9406431  0.5372014
##   12  0.9393406  0.5224839
##   13  0.9389287  0.5155935
##   14  0.9373694  0.4989574
##   15  0.9356978  0.4806254
##   16  0.9346803  0.4702331
##   17  0.9342495  0.4649525
##   18  0.9332643  0.4527168
##   19  0.9318786  0.4386231
##   20  0.9320203  0.4375610
## 
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 1.
```

Ggplot Observation

```r
ggplot(knn_tune$results, aes(x = k, y = Accuracy)) +
  geom_line() +
  geom_point() +
  geom_vline(xintercept = 5, linetype = "dashed", color = "red") +
  labs(title = "k-NN Accuracy vs. k (Bootstrap Resampling)",
       x = "k (number of neighbors)",
       y = "Accuracy") +
  theme_minimal()
```



Problem 2 Answer:

Although train() selected k=1 as having the highest bootstrapped accuracy, k=1 is susceptible to overfitting. Accuracy remains high and relatively stable through k=6-7, after which it begins a more consistent decline. Choosing k=6 sacrifices a small amount of accuracy (95.2% → 94.6%) in exchange for a model less sensitive to individual data points. To avoid a possible tie, testing k=5 directly against the validation and test sets, it outperformed k=6 on every metric, which supports settling on k=5 as the final choice.

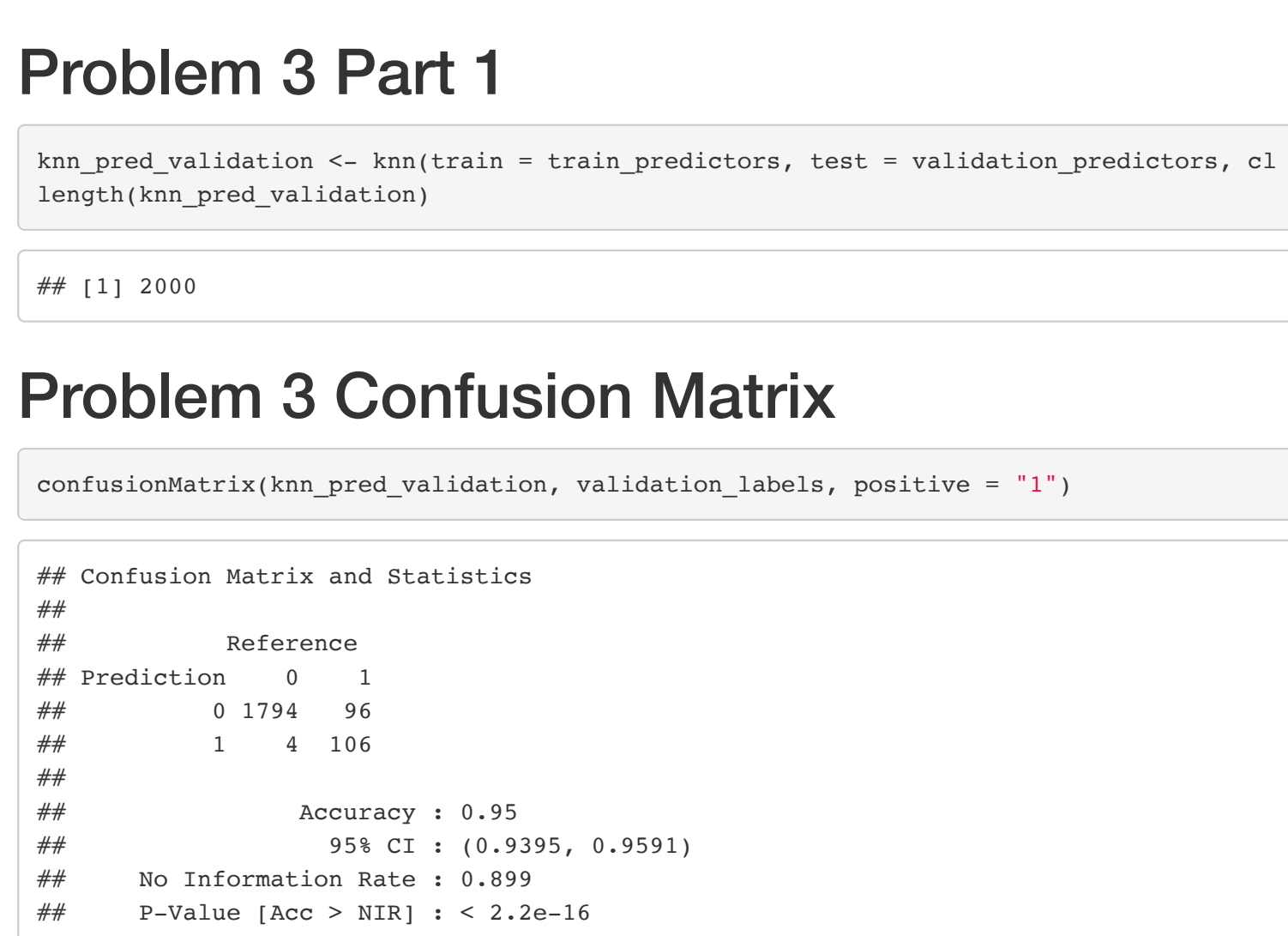
Validation Set Performance: k=6 vs k=5

```r
k_values <- c(1, 5, 6)
results_list <- list()

for (k_val in k_values) {
  pred <- knn(train = train_predictors, test = validation_predictors, cl = train_labels, k = k_val)
  cm <- confusionMatrix(pred, validation_labels, positive = "1")
  results_list[[as.character(k_val)]] <- c(
    Accuracy = cm$overall["Accuracy"],
    Sensitivity = cm$byClass["Sensitivity"],
    Specificity = cm$byClass["Specificity"],
    Kappa = cm$overall["Kappa"]
  )
}

comparison_df <- do.call(rbind, results_list)
comparison_df <- data.frame(k = k_values, comparison_df)
comparison_df
```

```
##   k Accuracy.Accuracy Sensitivity.Sensitivity Specificity.Specificity
## 1 1             0.962               0.7029703               0.9911012
## 5 5             0.950               0.5247525               0.9977753
## 6 6             0.946               0.4900990               0.9972191
##   Kappa.Kappa
## 1   0.7683520
## 5   0.6549106
## 6   0.6210420
```

Comparison Across K Values

```r
comparison_long <- pivot_longer(comparison_df,
                                cols = c(Accuracy.Accuracy, Sensitivity.Sensitivity,
                                         Specificity.Specificity, Kappa.Kappa),
                                names_to = "Metric", values_to = "Value")

ggplot(comparison_long, aes(x = factor(k), y = Value, fill = Metric)) +
  geom_col(position = "dodge") +
  labs(title = "Model Performance Comparison Across k Values",
       x = "k (number of neighbors)",
       y = "Score") +
  theme_minimal() +
  scale_fill_brewer(palette = "Set2")
```



Observation

Although k=1 is theoretically the most susceptible to overfitting, both the bootstrap resampling in the initial tuning step and a direct test on the held-out validation set show k=1 outperforming k=5 and k=6 across every metric, including accuracy (96.2%), sensitivity (70.3%), and Kappa (0.768). Given the size of this training set (3,000 observations), this suggests k=1 may generalize well here rather than overfitting to noise. However, because k=1's performance can be more sensitive to the particular train/validation split chosen, I've decided to stay with a more cautious choice k=5. For the remainder of this analysis, k=5 is used as the working choice, balancing performance with some margin against the instability k=1 can exhibit on unseen data

Problem 3 Part 1

```r
knn_pred_validation <- knn(train = train_predictors, test = validation_predictors, cl = train_labels, k = 5)
length(knn_pred_validation)
```

```
## [1] 2000
```

Problem 3 Confusion Matrix

```r
confusionMatrix(knn_pred_validation, validation_labels, positive = "1")
```

```
## Confusion Matrix and Statistics
## 
##           Reference
## Prediction    0    1
##          0 1794   96
##          1    4  106
##                                           
##                Accuracy : 0.95            
##                  95% CI : (0.9395, 0.9591)
##     No Information Rate : 0.899           
##     P-Value [Acc > NIR] : < 2.2e-16       
##                                           
##                   Kappa : 0.6549          
##                                           
##  Mcnemar's Test P-Value : < 2.2e-16       
##                                           
##             Sensitivity : 0.5248          
##             Specificity : 0.9978          
##          Pos Pred Value : 0.9636          
##          Neg Pred Value : 0.9492          
##              Prevalence : 0.1010          
##          Detection Rate : 0.0530          
##    Detection Prevalence : 0.0550          
##       Balanced Accuracy : 0.7613          
##                                           
##        'Positive' Class : 1               
## 
```

The k=5 model achieves 95.0% overall accuracy, with sensitivity (the ability to correctly identify actual loan-acceptors) at 52.5%, meaning the model still misses just under half of customers who would accept a loan offer, though this is an improvement over k=6 (49.0%). Specificity (99.78%) remains excellent on the non-acceptor class. Kappa (0.6549) also improved over k=6 (0.6162), indicating the k=5 model adds more value beyond chance once the class imbalance (Prevalence = 10.1%) is accounted for. Overall, k=5 outperformed k=6 on every metric in this comparison, not just sensitivity, suggesting it's a better-fitting neighborhood size for this dataset while also avoiding the tie-vote risk that comes with an even k.

Problem 4

```r
knn_result_q4 <- knn(train = train_predictors, test = new_customer_norm, cl = train_labels, k = 5)
knn_result_q4
```

```
## [1] 0
## Levels: 0 1
```

Using k=5, this customer is classified as 0 (loan not accepted). The same results were obtained under k=1 and k=6. Even though k values disagree on some customers overall (the k=5 validation confusion matrix shows 96 false negatives and 4 false positives, compared to 103/7 under k=6), all three choices of k agree on this particular customer's classification.

Problem 5

```r
set.seed(456)
train_index2 <- sample(1:nrow(bank_full), 0.50 * nrow(bank_full))
train_data2 <- bank_full[train_index2, ]
nrow(train_data2)
```

```
## [1] 2500
```

```r
remaining_data <- bank_full[-train_index2, ]
nrow(remaining_data)
```

```
## [1] 2500
```

```r
set.seed(456)
validation_index <- sample(1:nrow(remaining_data), 0.60 * nrow(remaining_data))
validation_data2 <- remaining_data[validation_index, ]
nrow(validation_data2)
```

```
## [1] 1500
```

```r
test_data2 <- remaining_data[-validation_index, ]
nrow(test_data2)
```

```
## [1] 1000
```

Normalization Step 1: Separate predictor from target for all three

```r
train_predictors2 <- train_data2[ , -which(names(train_data2) %in% c("Personal.Loan"))]
validation_predictors2 <- validation_data2[ , -which(names(validation_data2) %in% c("Personal.Loan"))]
test_predictors2 <- test_data2[ , -which(names(test_data2) %in% c("Personal.Loan"))]
```

Step 2: Build the recipe from train_predictors2 only, apply to all three

```r
norm_recipie2 <- preProcess(train_predictors2, method = c("range"))
train_predictors2 <- predict(norm_recipie2, newdata = train_predictors2)
validation_predictors2 <- predict(norm_recipie2, newdata = validation_predictors2)
test_predictors2 <- predict(norm_recipie2, newdata = test_predictors2)
```

Step 3: Build the labels for all three

```r
train_labels2 <- factor(train_data2$Personal.Loan)
validation_labels2 <- factor(validation_data2$Personal.Loan)
test_labels2 <- factor(test_data2$Personal.Loan)
```

```r
ncol(train_predictors2)
```

```
## [1] 13
```

Run k-NN predictions for each set

```r
knn_pred_train2 <- knn(train = train_predictors2, test = train_predictors2, cl = train_labels2, k = 5)
knn_pred_validation2 <- knn(train = train_predictors2, test = validation_predictors2, cl = train_labels2, k = 5)
knn_pred_test2 <- knn(train = train_predictors2, test = test_predictors2, cl = train_labels2, k = 5)
```

Build the three confusion matrices

```r
print("Training Set Confusion Matrix")
```

```
## [1] "Training Set Confusion Matrix"
```

```r
confusionMatrix(knn_pred_train2, train_labels2, positive = "1")
```

```
## Confusion Matrix and Statistics
## 
##           Reference
## Prediction    0    1
##          0 2256   87
##          1    3  154
##                                           
##                Accuracy : 0.964           
##                  95% CI : (0.9559, 0.971) 
##     No Information Rate : 0.9036          
##     P-Value [Acc > NIR] : < 2.2e-16       
##                                           
##                   Kappa : 0.7553          
##                                           
##  Mcnemar's Test P-Value : < 2.2e-16       
##                                           
##             Sensitivity : 0.6390          
##             Specificity : 0.9987          
##          Pos Pred Value : 0.9809          
##          Neg Pred Value : 0.9629          
##              Prevalence : 0.0964          
##          Detection Rate : 0.0616          
##    Detection Prevalence : 0.0628          
##       Balanced Accuracy : 0.8188          
##                                           
##        'Positive' Class : 1               
## 
```

```r
print("Validation Set Confusion Matrix")
```

```
## [1] "Validation Set Confusion Matrix"
```

```r
confusionMatrix(knn_pred_validation2, validation_labels2, positive = "1")
```

```
## Confusion Matrix and Statistics
## 
##           Reference
## Prediction    0    1
##          0 1348   82
##          1    3   67
##                                           
##                Accuracy : 0.9433          
##                  95% CI : (0.9304, 0.9545)
##     No Information Rate : 0.9007          
##     P-Value [Acc > NIR] : 1.819e-09       
##                                           
##                   Kappa : 0.5856          
##                                           
##  Mcnemar's Test P-Value : < 2.2e-16       
##                                           
##             Sensitivity : 0.44966         
##             Specificity : 0.99778         
##          Pos Pred Value : 0.95714         
##          Neg Pred Value : 0.94266         
##              Prevalence : 0.09933         
##          Detection Rate : 0.04467         
##    Detection Prevalence : 0.04667         
##       Balanced Accuracy : 0.72372         
##                                           
##        'Positive' Class : 1               
## 
```

```r
print("Test Set Confusion Matrix")
```

```
## [1] "Test Set Confusion Matrix"
```

```r
confusionMatrix(knn_pred_test2, test_labels2, positive = "1")
```

```
## Confusion Matrix and Statistics
## 
##           Reference
## Prediction   0   1
##          0 907  42
##          1   3  48
##                                          
##                Accuracy : 0.955          
##                  95% CI : (0.9402, 0.967)
##     No Information Rate : 0.91           
##     P-Value [Acc > NIR] : 3.838e-08      
##                                          
##                   Kappa : 0.6586         
##                                          
##  Mcnemar's Test P-Value : 1.473e-08      
##                                          
##             Sensitivity : 0.5333         
##             Specificity : 0.9967         
##          Pos Pred Value : 0.9412         
##          Neg Pred Value : 0.9557         
##              Prevalence : 0.0900         
##          Detection Rate : 0.0480         
##    Detection Prevalence : 0.0510         
##       Balanced Accuracy : 0.7650         
##                                          
##        'Positive' Class : 1              
## 
```

Conclusion

Using k=5, the training set again shows the highest accuracy (96.4%) and sensitivity (63.9%), since the model is evaluated on data it was built from some of its own points can act as their own nearest neighbors, inflating apparent performance. Validation (94.33%) and test (95.5%) show lower accuracy and notably lower sensitivity (44.97% and 53.33% respectively), reflecting more realistic performance on unseen data. Specificity remains consistently high (99.7–99.9%) across all three sets, reflecting that the model handles the majority non-acceptor class easily but struggles more with the minority acceptor class it's being evaluated on here. Compared to the earlier k=6 results, k=5 improved sensitivity in all three sets (training: 60%→63.9%, validation: 42%→44.97%, test: 49%→53.33%), supporting the choice of k=5 as a modest but genuine improvement, while also eliminating the possibility of tied votes that an even k like 6 could produce.